

Locks

2025 Semester 1 SOFTENG 370: Operating Systems
Talía Xu

Lecture 21
2.0.0

Recap From Yesterday

Concurrent access to a shared variable by multiple threads causes a data race.

Data race causes undefined behavior

A lock provides mutual exclusion
critical regions appear atomic to concurrent threads.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_lock(&lock); ←
```

```
// protected code
```

```
pthread_mutex_unlock(&lock); ←
```

Example of pthread lock

```
#define NUM_ITERATIONS 100000
int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void *worker(void *arg) {
    for (int i = 0; i < NUM_ITERATIONS; ++i) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
}
int main(void){
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL); pthread_join(t2, NULL);
    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Handwritten notes:

- 2* (next to `void *worker`)
- tv = counter* (with an arrow pointing to `int i = 0`)
- tv = tv + 1* (with an arrow pointing to `counter++`)
- counter = tv* (with an arrow pointing to `counter++`)
- W.* (next to the closing brace of `worker`)

You may need more than one lock

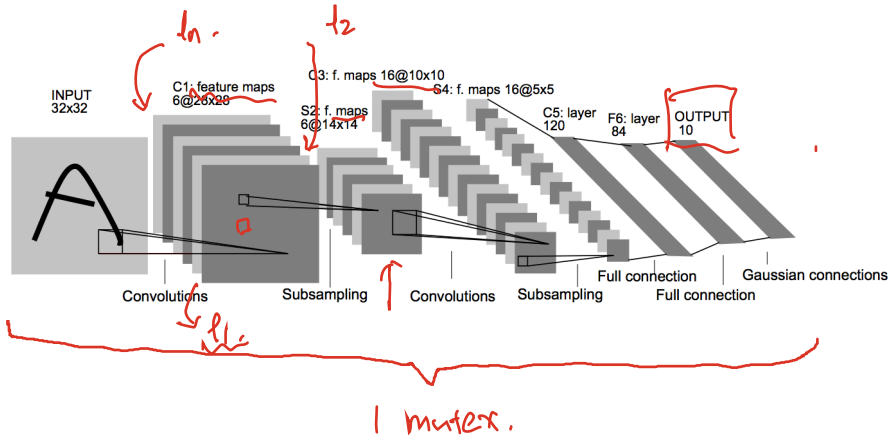
You can use one mutex for all shared data. Every thread must take the same lock before touching any of it.

Scales very poorly ↩

→ Or you can have each independent shared object (or field, or small group of fields) has its own mutex.

A thread grabs only the lock(s) that protect the data it will use.

Granularity of Locks - Example



An example with two counters

```
#define N_ITER 1000000
static long counterA = 0, counterB = 0;
static pthread_mutex_t lockA = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t lockB = PTHREAD_MUTEX_INITIALIZER;

void *incA(void *arg)
{
    for (int i = 0; i < N_ITER; ++i) {
        pthread_mutex_lock(&lockA);
        ++counterA;
        pthread_mutex_unlock(&lockA);
        pthread_mutex_lock(&lockB);
        ++counterB;
        pthread_mutex_unlock(&lockB);
    }
    return NULL;
}
```

What could happen with locks

```
pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
int main() {
    pthread_t x, y;
    pthread_create(&x, NULL, t1, NULL);
    pthread_create(&y, NULL, t2, NULL);
    pthread_join(x, NULL);
    pthread_join(y, NULL);
    return 0;
}
```

t1 →

```
void* t1(void* _) {
    pthread_mutex_lock(&a);
    pthread_mutex_lock(&b);
    pthread_mutex_unlock(&b);
    pthread_mutex_unlock(&a);
    return NULL;
}
```

→ v-a
→ v-b.

```
void* t2(void* _) {
    pthread_mutex_lock(&b);
    pthread_mutex_lock(&a);
    pthread_mutex_unlock(&a);
    pthread_mutex_unlock(&b);
    return NULL;
}
```

← t2.

A Critical Section Means Only One Thread Executes Instructions

- Safety (aka mutual exclusion)
There should only be a single thread in a critical section at once
- Liveness (aka progress)
If multiple threads reach a critical section, one must proceed ←
The critical section can't depend on outside threads
You can mess up and deadlock (threads don't make progress)
- Bounded waiting (aka starvation-free)
A waiting thread must eventually proceed

Critical Sections Should Also Have Minimal Overhead

- Efficient
You don't want to consume resources while waiting
- Fair
You want each thread to wait approximately the same time
- Simple
It should be easy to use, and hard to misuse

How do we build a lock?

How can we build an efficient lock?

You Could Use a Lock to Implement Critical Sections

Assuming a uniprocessor operating system, your implementation could be:

```
void lock() {  
    disable_interrupts();  
}  
void unlock() {  
    enable_interrupts();  
}
```

no timer
no I/O.

This would disable concurrency (assuming it ignores signals and interrupts)

This does not work on multiprocessors

Let's Try to Implement a Lock in Software

```
1 void init(int *l) {  
    *l = 0;  
}  
2 void lock(int *l) {  
    while (*l == 1);  
3    *l = 1;  
    }  
4 void unlock(int *l) {  
    *l = 0;  
    }
```

0: available.

btw ln 2 & 3.

a thread switch happens?

*l = 0.
t₁

t₂.

lock

completes 2.

lock

while ()

*l = ? 0. available.

l = 1, ←

Let's Try to Implement a Lock in Software

```
void init(int *l) {  
    *l = 0;  
}  
void lock(int *l) {  
    while (*l == 1);  
    *l = 1;  
}  
void unlock(int *l) {  
    *l = 0;  
}
```

← } 1 atomic

What's the issue with this implementation?

It's not safe (both threads can be in the critical section) *←*

~~~~~ It's not efficient, it wastes CPU cycles (busy wait)

Let's Assume a Magical Atomic Function — compare_and_swap

`compare_and_swap(int *p, int old, int new)` is atomic

It returns the original value pointed to

It only swaps if the original value equals `old`, and changes it to `new`

Let's give it another shot:

```
void init(int *l) {
```

```
    *l = 0;
```

```
}
```

```
void lock(int *l) {
```

```
    while (compare_and_swap(l, 0, 1)); ←
```

```
}
```

```
void unlock(int *l) {
```

```
    *l = 0;
```

```
}
```

< if $l=0$ > available.

c-s(l, 0, 1).

↳ `retval = 0`

$l = 1$

< if $l=1$ > (unavailable)

→ c-s(l, 0, 1)

↳ `retval = 1`

$l = 1$.

↓
spin-lock.

What We Implement is Essentially a Spinlock

- Compare and swap is a common atomic hardware instruction
On x86 this is the `cmpxchg` instruction (compare and exchange)
However, it still has this "busy wait" problem
- Consider a uniprocessor system, if you can't get the lock, you should yield
Let the kernel schedule another process, that may free the lock
- On a multiprocessor machine, you could try again

Let's Add a Yield

n. threads
→

```
void lock(int *l) {  
    while (compare_and_swap(l, 0, 1)) {  
        thread_yield(); → sleep.  
    }  
}
```

Multiple threads may be waiting on the same lock

We have no control over who gets the lock next

We need to be able to reason about it (FIFO is okay)

We Can Add a Wait Queue to the Lock

```
void lock(int *l) {  
    while (compare_and_swap(l, 0, 1)) {  
        // add myself to the lock wait queue  
        thread_sleep();  
    }  
}  
void unlock(int *l) {  
    *l = 0;  
    if (/* threads in wait queue */) {  
        // wake up one thread  
    }  
}
```

t_1 , t_2 .



Lost Wakeup Example

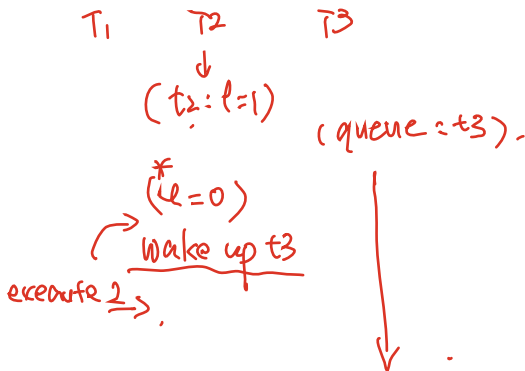
```
1 void lock(int *l) { ← t1. (→  
2 while (compare_and_swap(l, 0, 1)) {  
3   → // add myself to the wait queue  
4   thread_sleep(); ←  
5 }  
6 }  
7 void unlock(int *l) { ← t2  
8   *l = 0;  
9   if (/* threads in wait queue */) {  
10    // wake up one thread ↑  
11  }  
12 }
```

1. t2 calls unlock
t1 calls lock
(t2: l=1).
2. t1 ex. ln. 2.
↓ t2.
3. 8-12. exits unlock().
4. t1 ex. ln 4.
sleep(). (l is available)

Assume we have thread 1 (T1) and thread 2 (T2), thread 2 holds the lock
T1 runs line 2 and fails, swap to T2 that runs lines 10-12, T1 runs lines 3-4
T1 will never get woken up!

Wrong Thread Getting the Lock Example

```
1 void lock(int *l) {  
2   → while (compare_and_swap(l, 0, 1)) {  
3       // add myself to the wait queue  
4       thread_sleep();  
5   }  
6 }  
7 void unlock(int *l) {  
8     *l = 0;  
9     if (/* threads in wait queue */) {  
10        // wake up one thread  
11    }  
12 }
```



Assume we have T1, T2, and T3. T2 holds the lock, T3 is in queue.
T2 runs line 9, swap to T1 which runs line 2 and succeeds
T1 just stole the lock from T3!

We Can Use Two Variables to Fix This (One to Guard)

```
typedef struct { int lock; int guard;
                queue_t *q; } mutex_t;
```

busy waiting
guard = 0
g=0
l=1

```
void lock(mutex_t *m) {
    while (
        compare_and_swap(m->guard, 0, 1)
    );
    if (m->lock == 0) {
        m->lock = 1; // acquire mutex
        m->guard = 0;
    } else {
        enqueue(m->q, self);
        m->guard = 0;
        thread_sleep();
        // wakeup transfers the lock here
    }
}
```

t1

guard
lock
data

```
void unlock(mutex_t *m) {
    while (
        compare_and_swap(m->guard, 0, 1)
    );
    if (queue_empty(m->q)) {
        // release lock, no one needs it
        m->lock = 0;
    }
    else {
        // direct transfer mutex
        // to next thread
        thread_wakeup(dequeue(m->q));
    }
    m->guard = 0;
}
```

t2

There's **STILL** A Data Race

After a thread calls `lock`, it could get interrupted right before the `thread_sleep`

However, it's been added to the wait queue, so `thread_wakeup` ←
would try to wake up a thread that's not sleeping yet (we know it's about to)

We could simply retry the call to `thread_wakeup` ← .
until the thread finally calls `thread_sleep`